

category: linux-desktop

tags: [wayland, gnome, mutter, dbus, pipewire, gstreamer, remote-control, kiosk, systemd]

severity: high

created: 2026-07-31

updated: 2026-07-31

project-origin: IPGS.RemoteControl.ZcuAgent (branch wayland)

Remote control trên GNOME Wayland (kiosk không người trực): Mutter D-Bus private API + GStreamer subprocess, KHÔNG dùng xdg-desktop-portal chuẩn

Tình huống gặp phải

`IPGS.RemoteControl.ZcuAgent` vốn chỉ chạy trên X11 (XShm capture + XTest input), refuse khởi động cứng nếu `XDG_SESSION_TYPE != x11`. Cần thêm nhánh Wayland cho kiosk GNOME Shell 42 (Ubuntu 22.04) chạy KHÔNG có người trực — không ai ngồi bấm "Allow" trên dialog xin chia sẻ màn hình.

Quyết định kiến trúc quan trọng: Mutter private API, không phải xdg-desktop-portal

- **xdg-desktop-portal chuẩn** (`org.freedesktop.portal.ScreenCast/RemoteDesktop`) là API cross-desktop, ổn định lâu dài, nhưng LUÔN hiện dialog xin xác nhận chia sẻ màn hình mỗi phiên (trừ khi lưu restore token, và ngay cả restore token vẫn cần 1 lần đồng ý ban đầu). Không dùng được cho kiosk chạy tự động.

- **API riêng của Mutter** (`org.gnome.Mutter.ScreenCast + org.gnome.Mutter.RemoteDesktop`, cùng API mà `gnome-remote-desktop` — tính năng VNC/RDP built-in của GNOME — dùng nội bộ) KHÔNG hiện dialog vì process gọi chạy trong CÙNG session user (không sandbox/Flatpak). Đánh đổi: đây là API riêng/không ổn định của Mutter, có thể lệch giữa các bản GNOME.

Quyết định: dùng Mutter private API vì kiosk không người trực loại bỏ hoàn toàn lựa chọn portal chuẩn — đã hỏi user xác nhận trade-off này trước khi code (xem thêm ghi chú "cần verify trên máy thật" bên dưới, vì KHÔNG thể test API D-Bus này từ máy dev Windows).

Gotcha 1: Pairing RemoteDesktop + ScreenCast phải đúng thứ tự

Để ScreenCast session được Mutter tự cấp quyền (không bị coi là request độc lập), **PHẢI**:

1. `RemoteDesktop.CreateSession()` trước → lấy `SessionId`.
2. `ScreenCast.CreateSession({"remote-desktop-session-id": <SessionId>})` — property này bắt buộc để ghép cặp 2 session.
3. Subscribe signal `Stream.PipeWireStreamAdded` TRƯỚC KHI gọi `Start()` trên cả 2 session — signal có thể bắn ngay sau `Start()`, subscribe sau dễ miss.

Gotcha 1b: ĐÃ VERIFY TRÊN MÁY THẬT (192.168.21.230, GNOME Shell 42.9, 2026-07-31) — `ScreenCastSession.Start()/Stop()` KHÔNG được gọi trực tiếp khi đã ghép cặp

Verify bằng script PyGObject (`Gio.DBusConnection`, giữ 1 connection sống xuyên suốt — quan trọng vì mỗi lệnh `busctl call/gdbus call` riêng lẻ tự mở rồi ĐÓNG connection ngay sau khi trả kết quả, mà session object của Mutter bị **hủy ngay khi peer connection tạo ra nó ngắt kết nối** — dùng `busctl` gọi nhiều lệnh rồi rạc sẽ luôn thấy "Object does not exist" ở lệnh thứ 2 trở đi, trông giống session tự hết hạn nhưng thực ra là do tool CLI tự đóng kết nối, không phải do Mutter timeout).

Kết quả verify quan trọng nhất — khác với giả định ban đầu trong code:

```
RemoteDesktopSession.Start() → OK, đồng thời TỰ ĐỘNG start luôn ScreenCast đã ghép cặp
                               (PipeWireStreamAdded bắn ra ngay sau, không cần gọi
                               thêm gì)
ScreenCastSession.Start()    → GDBus.Error: "Must be started from remote desktop
session"
ScreenCastSession.Stop()     → GDBus.Error: "Must be stopped from remote desktop
session"
```

→ Khi ScreenCast session được tạo kèm property `remote-desktop-session-id`, **CHỈ được gọi `RemoteDesktopSession.Start()/Stop()` — gọi `ScreenCastSession.Start()/Stop()` trực tiếp là lỗi runtime chắc chắn. Code ban đầu viết cả 2 lời gọi (trông là cần thiết theo suy luận từ tài liệu chung chung) — nếu không test trên máy thật sẽ KHÔNG BAO GIỜ phát hiện ra, vì:

- Build/compile không thể bắt lỗi này (đây là runtime D-Bus error, không phải type error).
- Introspection (`busctl --user introspect .../Session/uN`) trả về **node XML RỖNG** — GNOME

Shell (viết bằng GJS) không tự sinh introspection cho các interface private này, nên không có cách nào đọc trước danh sách method/signature hợp lệ mà không thử gọi thật.

Gotcha 2: Capture PipeWire không dùng native binding — dùng subprocess GStreamer

Đàm phán format/buffer PipeWire trực tiếp qua P/Invoke libpipewire là bề mặt lớn, ít tài liệu, dễ sai. Giải pháp thực dụng: shell ra `gst-launch-1.0` với `pipewiresrc path=<node_id>`. Cần thêm `videorate` để GStreamer tự throttle FPS ở nguồn — nếu không, monitor sinh frame ở fps gốc (thường 60) trong khi agent chỉ tiêu thụ ở 15fps, làm OS pipe buffer đầy và tăng

latency dần theo thời gian.

Cách lấy resolution mà không cần biết trước: parse `width=(int)N, height=(int)M` từ

verbose caps negotiation log (`gst-launch-1.0 -v`) bằng regex, với timeout — không có API rõ ràng khác để biết resolution trước khi frame đầu tiên tới.

Gotcha 2b: ĐÃ VERIFY TRÊN MÁY THẬT — `-v` in ra STDOUT chứ không phải stderr, làm hỏng dữ liệu pixel nếu dùng chung `fdsink fd=1`

Thiết kế ban đầu: `pipewiresrc ... ! fdsink fd=1` (ghi pixel ra stdout của process) + parse caps từ **stderr** (giả định `-v` in ra stderr như log level thông thường). Chạy thật trên máy: file output toàn ASCII text bắt đầu bằng `"Pipeline is live and does not need PREROLL..."` — `**verbose text` của `-v` in ra **STDOUT****, trộn lẫn với binary pixel data cùng kênh, hỏng hoàn toàn dữ liệu. file `/tmp/out.raw` báo "ASCII text" thay vì binary là dấu hiệu nhận biết.

Fix: tách 2 kênh vật lý — text verbose vẫn đọc qua `Process.StandardOutput` (`RedirectStandardOutput`), còn pixel data ghi qua **named pipe (FIFO)** bằng `filesink location=<fifo>` thay vì `fdsink fd=1.mkfifo` tạo trước khi start process, C# mở `FileStream` đọc riêng.

⚠ **Gotcha lồng bên trong fix này — nguy cơ deadlock rendezvous:** mở FIFO để ĐỌC blocks cho đến khi có bên GHI kết nối; `filesink` mở FIFO để GHI cũng blocks cho đến khi có bên ĐỌC kết nối. Nếu code đợi tuần tự "parse xong caps rồi mới mở FIFO đọc", có thể deadlock nếu `filesink` cần mở FIFO (đợi reader) TRƯỚC KHI in nốt caps message cuối. Giải pháp: chạy việc "mở FIFO đọc" và "đợi parse caps" CONCURRENT (`Task.Run` riêng, không chờ tuần tự), rồi `await/.Wait()` cả hai.

Gotcha 3: Input trên Mutter khác hẳn XTest — ĐÃ VERIFY TRÊN MÁY THẬT (evdev/axis/keysym đúng, nhưng D-Bus signature khác giả định ban đầu)

- Mouse button dùng **evdev code** (`BTN_LEFT=0x110, BTN_RIGHT=0x111, BTN_MIDDLE=0x112`), KHÔNG phải X11 button number (1/2/3). `NotifyPointerButton(int, bool)` verify đúng — chú ý tham số PHẢI là `int` (**signed**), gọi với `uint` báo lỗi `InvalidArgs: expected "(ib)"`.
- Mouse wheel là **pointer axis event** (`NotifyPointerAxisDiscrete(uint, int)`), không phải button press/release như XTest — chỉ bắn 1 step ở cạnh nhấn, bỏ qua cạnh nhả. Signature `(ui)` verify đúng khớp.
- Keyboard nhận thẳng X11 keysym qua `NotifyKeyboardKeysym(uint, bool)` — Mutter tự resolve qua xkb nội bộ, ĐƠN GIẢN HƠN X11 (không cần `XKeysymToKeycode`). Verify đúng khớp hoàn toàn.
- ⚠ `NotifyPointerMotionAbsolute` — **BUG THẬT phát hiện khi verify:** tham số đầu (stream identifier) tưởng là `ObjectPath` (kiểu `o`) vì giá trị nhìn giống object path (`/org/gnome/Mutter/ScreenCast/Stream/uN`) — thực tế signature là `(sdd)`, tham số đầu là **string thường**, không phải object path (`InvalidArgs: "(odd)" does not match expected`

type "(sdd)". Truyền `streamPath.ToString()` thay vì object `ObjectPath` trực tiếp. Đây là gotcha kinh điển: D-Bus phân biệt kiểu `o` (object path) và `s` (string) ở tầng wire dù giá trị hiển thị giống hệt nhau — không có cách nào đoán đúng ngoài thử thật hoặc introspection (mà introspection ở đây lại rỗng, xem Gotcha 1b).

Gotcha 5: Tmds.DBus 0.92+ (Tmds.DBus.Emit codegen) yêu cầu interface PUBLIC, không được `internal`

Khai interface D-Bus proxy là `internal` (theo convention chung của codebase — mọi service class khác đều `internal`) khiến runtime throw ngay khi gọi `CreateProxy<T>`:
`TypeLoadException: Type '...Proxy' from assembly 'Tmds.DBus.Emit' is attempting to implement an inaccessible interface`. Tmds.DBus bản 0.92+ sinh proxy trong MỘT ASSEMBLY DYNAMIC RIÊNG (`Tmds.DBus.Emit`), không có quyền truy cập `internal` của assembly chứa interface — phải để `public`. Lỗi này KHÔNG xuất hiện lúc build (chỉ là access modifier hợp lệ về mặt C#), chỉ nổ ra khi `Connection.CreateProxy<T>()` chạy thật lúc runtime trên máy Linux thật.

Gotcha 6: Tmds.DBus — property D-Bus phải khai qua `GetAllAsync()`, không phải method `GetXAsync()`

Khai `Task<string> GetSessionIdAsync();` cho property D-Bus `SessionId` (đọc qua `org.freedesktop.DBus.Properties.GetAll`) — Tmds.DBus map thẳng tên method (bỏ hậu tố Async) thành lệnh gọi D-Bus METHOD `GetSessionId`, không tự nhận ra đây là ý định đọc property. Lỗi:
`UnknownMethod: No such method "GetSessionId".` Fix: khai `Task<IDictionary<string,object>> GetAllAsync();`, gọi rồi đọc key `"SessionId"` từ dictionary trả về.

Gotcha 4: NuGet Tmds.DBus — advisory GHSA-xrw6-gwf8-vvr9 chỉ vá ở 0.92.0 cho package chính

`dotnet build` báo NU1903 (high severity) cho MỌI version pre-0.90 (0.15.0 → 0.23.0 đều dính). Advisory ghi "fixed in 0.92.0" nhưng cũng ghi "backported to 0.21.3 for Tmds.DBus.Protocol" — **0.21.3 là version của package KHÁC** (`Tmds.DBus.Protocol`, low-level), KHÔNG PHẢI của `Tmds.DBus` (package classic API, `CreateProxy<T>` reflection). Để nhằm đọc advisory rồi pin nhằm version vẫn dính lỗi. Phải pin `Tmds.DBus >= 0.92.0` — May mắn là 0.92.0 vẫn giữ `CreateProxy<T>` API cũ (nuspec description "uses reflection and runtime code generation" không đổi qua version), không cần viết lại code khi nâng từ 0.15→0.92.

Cài đặt lệnh: systemd unit hardcoded `XDG\_SESSION\_TYPE=x11` phá luôn Wayland detection

`ZcuRemoteInstallerService.cs` (bộ cài qua SSH) tạo systemd `--user` unit với `Environment=XDG_SESSION_TYPE=x11` HARDCODE — dù agent code đã detect đúng theo env thật,

installer ép cứng x11 khiến agent LUÔN nghĩ mình đang X11 kể cả khi máy thật chạy Wayland, crash khi gọi `XOpenDisplay` (không có X server). Root cause: `XDG_SESSION_TYPE` KHÔNG tự inherit vào systemd user unit như các biến khác — phải set thủ công đúng session type, và trước đó không ai để ý vì lúc đó agent CHỈ hỗ trợ x11 nên hardcode "vô hại". Thêm nhánh Wayland thì hardcode này thành bug tiềm ẩn — phải sửa installer để dò session type thật (qua `loginctl show-session <sid> -p Type`, KHÔNG phải `echo $XDG_SESSION_TYPE` qua SSH vì phiên SSH luôn headless/rỗng) và set động vào unit file.

Gotcha 7: ĐÃ VERIFY TRÊN MÁY THẬT (192.168.21.96) — race condition lúc boot: systemd user service khởi động TRƯỚC KHI org.gnome.Shell đăng ký D-Bus name

Sau khi reboot máy kiosk thật, `ipgs-remote-agent.service` (`After=graphical-session.target`) khởi động và crash-loop **4 lần liên tiếp** (mỗi lần core-dump) trong ~20 giây đầu, với lỗi:

```
Tmds.DBus.DBusException: org.freedesktop.DBus.Error.ServiceUnknown:  
The name org.gnome.Shell was not provided by any .service files
```

`graphical-session.target` chỉ đảm bảo có PHIÊN ĐỒ HOẠ, KHÔNG đảm bảo GNOME Shell đã đăng ký xong D-Bus name `org.gnome.Shell` — có độ trễ thực tế ~20s giữa 2 sự kiện này. Đây rất có thể là nguyên nhân triệu chứng người dùng báo "cài được nhưng màn hình đen" — CCU kết nối đúng lúc agent đang crash-loop.

Fix: thêm retry loop TRONG `MutterSessionManager.StartAsync()` — bắt riêng

`DBusException` có `ErrorMessage == "org.freedesktop.DBus.Error.ServiceUnknown"`, retry mỗi 2s tối đa 60s thay vì để exception unhandled làm process abort (core-dump, tốn CPU/disk). Tốt hơn nhiều so với dựa vào `systemd Restart=on-failure` — restart bên ngoài process vẫn crash-loop ồ ạt, còn retry bên trong xử lý êm, không core-dump.

Gotcha 8: ĐÃ VERIFY TRÊN MÁY THẬT — lỗi gõ nhầm CIDR trong config làm agent từ chối MỌI kết nối im lặng

`appsettings.json` do CCU Setup Wizard ghi ra có `"AllowedClientIPs": ["0.0.0..0/0"]` — **2 dấu chấm** giữa `0.0.0` và `0/0` (lỗi gõ tay khi nhập vào wizard). CIDR sai định dạng này không parse được trong `AuthManager.IsInRange` → theo thiết kế deny-by-default (F06, xem `AgentOptions/ValidateSecurityConfig`), MỌI client bị từ chối — CCU kết nối TCP thành công (thấy cửa sổ Remote Control mở ra) nhưng bị disconnect ngay ("Đã ngắt kết nối") vì auth/IP check fail, và màn hình chưa kịp nhận frame nào nên hiện đen. Triệu chứng dễ nhầm với lỗi capture Wayland, nhưng thực ra là lỗi data-entry ở tầng config — luôn kiểm tra `appsettings.json` thật trên máy trước khi nghi ngờ code capture khi gặp "kết nối được rồi mất ngay + màn hình đen".

Gotcha 9: ĐÃ VERIFY TRÊN MÁY THẬT — Wayland (GStreamer/PipeWire/FIFO) giật/trễ hơn rõ rệt so với X11 (XShm pull), ngay cả sau khi thêm leaky queue

So sánh trực tiếp trên cùng 1 kiosk: nhánh X11 (pull trực tiếp qua `XShmGetImage`) mượt, không giật. Nhánh Wayland (dù đã thêm `queue leaky=downstream` để chống tích lũy backlog, và đã sửa `TargetFps` đọc đúng từ config) vẫn giật/trễ hơn rõ rệt theo cảm nhận người dùng thực tế.

Nguyên nhân kiến trúc (không phải bug, là chi phí cố hữu của thiết kế): đường Wayland có NHIỀU lớp IPC hơn X11 — mỗi frame phải đi qua: PipeWire (compositor → pipewiresrc) → subprocess `gst-launch-1.0` (videoconvert/videorate/queue) → named pipe FIFO (kernel) → .NET đọc `FileStream` → JPEG encode → TCP. X11 chỉ có: `XShmGetImage` (shared memory, zero-copy gần như) → JPEG encode → TCP. Số lớp context-switch/copy nhiều hơn hẳn.

Áp dụng lại: khi so sánh hiệu năng 2 nhánh, đừng kỳ vọng Wayland đạt độ mượt ngang X11 với kiến trúc subprocess-GStreamer này — đây là trade-off đã chấp nhận khi chọn hướng "không viết native libpipewire binding" (xem Gotcha 2). Nếu độ mượt là ưu tiên cao nhất và đội ngũ có thời gian đầu tư, hướng cải thiện tiếp theo (chưa làm) là thay subprocess GStreamer bằng native libpipewire binding trực tiếp (bỏ được lớp subprocess + FIFO), nhưng đánh đổi lại là bề mặt P/Invoke lớn hơn nhiều như đã phân tích ở Gotcha 2.

Áp dụng lại (How to reuse)

- Kiosk không người trực trên GNOME Wayland cần capture/input → luôn cân nhắc Mutter private API thay vì `xdg-desktop-portal`, NHƯNG phải verify method/signal signature bằng `busctl --user introspect org.gnome.Shell /org/gnome/Mutter/{ScreenCast,RemoteDesktop}` trên đúng bản GNOME mục tiêu trước khi rollout — đây là API không ổn định giữa các version.
- PipeWire capture trong .NET: subprocess GStreamer (`gst-launch-1.0`) là lựa chọn thực dụng hơn viết P/Invoke libpipewire trực tiếp, miễn là thêm `videorate` để throttle ở nguồn.
- Khi nâng version NuGet để vá lỗ hổng bảo mật: đọc kỹ advisory xem fix áp dụng cho ĐÚNG package đang dùng hay một package liên quan cùng repo (để nhầm khi có nhiều package cùng tên gốc như `Tmds.DBus` / `Tmds.DBus.Protocol`).
- Bất kỳ biến môi trường nào agent tự detect lúc runtime (`XDG_SESSION_TYPE`, `DISPLAY`, ...) — nếu có installer/deploy script tạo systemd unit, PHẢI set biến đó ĐỘNG theo giá trị dò được, KHÔNG hardcoded, kể cả khi hiện tại chỉ có 1 giá trị khả dĩ (dễ bug tiềm ẩn khi mở rộng sau này).

Chú ý / Cạm bẫy (Gotchas)

- ☒ **ĐÃ VERIFY ĐẦY ĐỦ END-TO-END trên máy thật** (192.168.21.230, GNOME Shell 42.9, 2026-07-31): D-Bus session pairing, `ScreenCastSession` KHÔNG gọi Start/Stop riêng, capture PipeWire qua GStreamer FIFO (1080×1920, agent chạy sạch, TCP server listen đúng port), và cả 4 lệnh input (`NotifyPointerMotionAbsolute`, `NotifyPointerButton`,

`NotifyPointerAxisDiscrete, NotifyKeyboardKeysym`) — sau khi fix 4 bug thật (Gotcha 1b, 2b, 3, 5, 6) phát hiện CHỈ QUA chạy thật, không phát hiện được bằng build/compile hay introspection D-Bus (node XML rỗng, xem Gotcha 1b).

- Build đã publish binary + đóng gói vào `IPGS.RemoteControl.CcuUI/Resources/zcu-agent/linux-x64/` (offline install, cùng pattern `x11-deb/dotnet-runtime` có sẵn).
- ⚠ `echo $XDG_SESSION_TYPE` qua kết nối SSH luôn trả rỗng/headless — phải dò qua `loginctl show-session` để lấy session type của phiên đồ họa thật đang chạy.
- ⚠ Đọc security advisory NuGet: chú ý advisory có thể liệt kê version vá cho NHIỀU package liên quan (repo mono-repo kiểu `Tmds.DBus/Tmds.DBus.Protocol`) — xác nhận version áp dụng cho đúng package đang `PackageReference`.

Tham chiếu

- Project: `IPGS.RemoteControl.ZcuAgent` (branch `wayland`) —
`Wayland/MutterDBusInterfaces.cs`,
`Wayland/MutterSessionManager.cs`, `Wayland/WaylandScreenCapturer.cs`,
`Wayland/WaylandMouseInjector.cs`, `Wayland/WaylandKeyboardInjector.cs`
- TDD: `docs/tech-design/TDD-remote-control.md §14b`
- Deploy: `docs/devops/DEPLOY-remote-control.md §2.1/2.3`
- GHSA-xrw6-gwf8-vvr9 / CVE-2026-39959 (Tmds.DBus)
- Related lesson: `camera-integration/x11-xshmattach-async-baderror-crashes-process.md` (cùng project, nhánh X11)